PRODUCT REQUIREMENTS DOCUMENT Backend System – KreaSea AI Content Studio Versi: 1.0 **Tanggal:** 12 Mei 2026 **Author:** Grok (Academic Success Partner & Full-Stack Consultant) **Status:** Final – Siap Implementasi **Cakupan:** Seluruh backend (API Gateway + Services) untuk mendukung Flutter App, Admin Dashboard, dan fitur tambahan v1.0 (PRD Fitur Tambahan).

Catatan Kritis dari Analisis Saya Arsitektur saat ini (dari diagram KreaSea) sudah solid: API Gateway + modular Backend Services + Firebase + multi-AI providers. Namun, ada **kelemahan potensial**:

- Single API key per provider → rentan rate-limit & downtime.
- Semua AI call **wajib** melalui backend (tidak boleh ada key di Flutter).
- Codebase harus dirancang agar scalable ke 10.000+ user tanpa rewrite besar.
- Belum ada orchestrator untuk auto-switch AI key/provider.

Saya rancang sistem ini agar **secure by default**, **maintainable**, dan **future-proof**. Semua saran di bawah ini actionable dan selaras dengan Panduan KP + PRD Fitur Tambahan.

---

1. TUJUAN BACKEND

- Menjadi **single entry point** yang aman untuk semua client (Flutter, Admin Dashboard, Bio Link pages).
- Mengintegrasikan **multi-AI providers** dengan intelligence (auto-switch key jika limit/quota habis).
- Menjamin **keamanan data user & API key** (tidak pernah bocor ke client).
- Mendukung **scalability** horizontal & maintainability tinggi.
- Memenuhi standar akademik KP (dokumentasi lengkap, observability, error handling).

---

2. ARSITEKTUR BACKEND (Proposed – Improvement dari Diagram Existing)

High-Level Diagram (Text-based)

text

```
Client (Flutter / Admin Dashboard)
    ↓ HTTPS + JWT
API Gateway (Node.js / FastAPI - single entry)
    ↓ (Rate limit, Auth, Validation, Prompt Orchestration)
Backend Services (Micro-services pattern via modules)
    ├── Auth & Profile Service
    ├── Content AI Service (Gemini/Claude/GPT)
    ├── Image Generation Service (Stability + fallback)
```

```
    ├── Business Tools Service (HPP, GMaps, etc.)
    ├── Content Planner & Library
    ├── Quota & Billing
    ├── Notification & Bio Link
    └── Admin & Analytics
        ↓
AI Orchestrator Layer (baru – wajib)
    ├── Multi-Key Manager (rotation + fallback)
    ├── Caching (Redis)
    └── External Providers (Gemini, Stability, Claude, GPT-4.5)
Firebase (Auth, Firestore, Storage, FCM) + Midtrans/Xendit + Cloudflare CDN
```

Teknologi Stack Rekomendasi (Improvement)

Layer	Teknologi Rekomendasi	Alasan (Critical)	Alternatif
API Gateway	FastAPI (Python)	Async, auto OpenAPI docs, built-in rate limit & dependency injection	Node.js (jika tim lebih nyaman)
Backend Services	FastAPI modular (atau NestJS)	Type safety, easy testing, scalable	Node.js + Express
AI Orchestrator	Dedicated FastAPI module	Centralisasi prompt & key rotation	-
Database	Firebase Firestore + Redis	Realtime + caching performa tinggi	PostgreSQL (future)
Queue	Redis + Celery / BullMQ	Background job (quota, analytics)	-
Observability	Sentry + Prometheus + Grafana	Wajib untuk production & KP dokumentasi	-
Deployment	Docker + Docker Compose → Railway / Fly.io / GCP	Mudah scale & CI/CD	Vercel (kurang optimal untuk AI)

Mengapa FastAPI? Lebih baik untuk AI-heavy workload (async, Pydantic validation, background tasks).

---

3. MULTI-API KEY MANAGEMENT (Fitur Baru – Prioritas Tinggi)

Requirements

- Support **multiple keys per provider** (minimal 3–5 key per AI).
- **Auto-switch** jika rate limit / quota habis / error 429/5xx.
- Semua key disimpan **hanya di backend** (environment variables + encrypted secret manager).

- Logging rotasi tanpa expose key ke client.

Data Model (Firestore + Redis)

JSON

```
// Collection: ai_providers
{
  "provider": "gemini",
  "keys": [
    { "key_id": "g1", "value": "encrypted_key", "status": "active",
"quota_remaining": 95000, "last_used": timestamp },
    { "key_id": "g2", ... }
  ],
  "priority_order": [ "g1", "g2", "g3" ],
  "fallback_provider": "claude"
}
```

Alur AI Orchestrator (Pseudocode Logic)

1. Client panggil /api/content/generate → Gateway validasi JWT + quota user.
2. Orchestrator ambil key aktif (dari Redis cache).
3. Kirim request ke AI provider.
4. Jika error 429/ quota habis → tandai key tersebut cooldown, ambil key berikutnya, retry (max 3x).
5. Jika semua key provider gagal → switch ke fallback provider otomatis.
6. Update quota di Redis + Firestore (background task).

Acceptance Criteria

- Zero exposure API key ke Flutter.
- Retry & fallback < 5 detik.
- Dashboard Admin bisa lihat usage per key (tanpa tampilkan key value).

---

4. SECURITY & COMPLIANCE (Critical Must-Have)

- **Authentication**: Firebase Auth + custom JWT (short-lived 15 menit).
- **Authorization**: Role-based (user, admin) + Firestore Rules ketat.
- **Rate Limiting**: Per user (10 req/min free, higher di Pro) + global.
- **Input Validation & Sanitization**: Pydantic + OWASP rules.
- **API Key Shielding**: Semua AI call via backend.
- **Data Protection**: Encrypt sensitive fields (e.g. bio link stats).
- **CORS + HTTPS**: Cloudflare enforced.
- **Audit Log**: Semua action penting dicatat (Sentry + Firestore).

Risk Mitigation: Implementasi Security Rules Firebase + backend validation ganda.

---

5. SCALABILITY & MAINTAINABILITY

- **Modular Services**: Setiap service di folder terpisah (e.g. services/content_ai/).
- **Versioning API**: /api/v1/...
- **Caching**: Redis untuk prompt response yang sama & AI quota.
- **Background Jobs**: Celery/RQ untuk quota reset, analytics, email.
- **Testing**: 80%+ coverage (unit + integration).
- **Documentation**: Auto-generated OpenAPI + Swagger UI.

Codebase Best Practices

- Gunakan **Clean Architecture** / Domain-Driven Design.
- Environment variables via .env + Docker secrets.
- Git flow + conventional commits.
- Pre-commit hooks (black, isort, pylint).

---

6. API DESIGN STANDARDS (Contoh Endpoint Penting)

Semua endpoint gunakan REST + JSON.

Endpoint	Method	Deskripsi	Auth	Rate Limit
/api/auth/profile	GET/POST	Profil user + quota	JWT	30/min
/api/content/generate	POST	Caption / planner	JWT	10/min (free)
/api/image/generate	POST	Image via Stability	JWT	5/min
/api/hpp/calculate	POST	Kalkulator HPP	JWT	20/min
/api/ai/orchestrator/status	GET	Admin only – key usage	Admin	-

Request/Response selalu pakai standardized wrapper:

JSON

```
{ "success": true, "data": {...}, "error": null }
```

---

7. ERROR HANDLING, LOGGING, MONITORING

- Centralized error handler (FastAPI middleware).
- Graceful fallback untuk semua AI call.
- Logging: Structured JSON ke Sentry + Firebase Analytics.
- Alerting: Sentry untuk error > 5% atau quota critical.

---

8. DEPLOYMENT & CI/CD

- Docker + Docker Compose (dev).
- Production: Railway / Fly.io / GCP Cloud Run (auto-scale).
- CI/CD: GitHub Actions (test → build → deploy).
- Environment: dev, staging, production.

---

9. RISIKO & MITIGASI (Critical Analysis)

Risiko	Dampak	Mitigasi	Prioritas
AI cost meledak	Tinggi	Multi-key + quota hard limit + caching	High
Backend downtime	Tinggi	Auto-scaling + fallback providers	High
Data breach	Critical	No key di client + encryption	High
Scope creep fitur tambahan	Medium	Freeze fitur KP di Minggu 10	Medium

---

10. NEXT STEPS (Actionable Checklist untuk Kamu)

1. **Minggu ini**: Setup FastAPI skeleton + API Gateway + AI Orchestrator (multi-key).
2. **Minggu depan**: Implementasikan 3 endpoint inti (caption, image, HPP).
3. **Sebelum KP mulai**: Deploy staging + Sentry.
4. **Dokumentasi**: Buat backend/README.md + OpenAPI spec.